The Verification Problem in Compute Governance

April 2026, BlueDot Technical AI Safety Project

Aline Gabriel de Almeida

TL;DR:

Many proposals for governing advanced AI rely on monitoring large-scale training, but in practice it may be difficult to verify what GPUs are doing without intrusive access to proprietary software or data.

This project examines whether hardware-level telemetry preserves information about how computation is organized over time. Initial comparisons suggest that different workloads — including LLM training, LLM inference, CNN training (ResNet50), and masked language modeling (BERT) — can produce distinguishable temporal patterns in these signals.

These results do not solve the verification problem, but they suggest that non-intrusive hardware signals may offer a promising source of evidence about how compute is being used, without requiring access to sensitive model internals.

The problem

Large language models (LLMs) are trained through large-scale computation on GPUs. In recent years, scaling up these training runs has been one of the most reliable ways to build more capable AI systems. As a result, frontier AI development depends not only on data and algorithms, but also on access to large amounts of compute.

Because of this, more capable AI systems are likely to have very large effects on the world. They could generate substantial benefits, but they could also increase risks: enabling misuse at greater scale, making failures harder to control, and concentrating power in the hands of a small number of actors. If AI capabilities continue to advance, the ability to shape who can train frontier systems, and under what conditions, may become increasingly important.

This naturally points to compute as a target for governance. Since large-scale training is one of the clearest drivers of frontier model capability, many proposals focus on monitoring or restricting the compute used to develop advanced systems. However, the same GPUs used to train an LLM can also be used for entirely different workloads, making it difficult to externally infer what that compute is actually being used for.

Additionally, monitoring may not be acceptable in practice. Many approaches rely on direct access to systems, logs, or model internals, which can raise concerns about data security, intellectual property, or operational risk. This introduces a practical constraint: monitoring must not only be effective, but also compatible with real-world incentives and deployment conditions.

In particular, approaches that require modifying workloads or introducing new software components may be harder to adopt, especially for sensitive or strategic systems. They may introduce new risks, such as backdoors or unintended vulnerabilities, which could discourage GPU owners from participating in monitoring schemes.

Without credible and practical ways to verify when large-scale training is actually taking place, compute governance faces a fundamental enforcement problem. Rules may exist on paper, but without reliable verification, it becomes difficult to know whether they are being followed in practice.

Why this matters

This verification gap has direct implications for which forms of AI governance are feasible in practice.

If large-scale training cannot be observed externally, many policy approaches would need to rely on voluntary reporting or internal monitoring. While these can play a role, they may be difficult to verify, especially in competitive or low-trust environments.

In such settings, enforcement becomes more fragile. It may be harder to detect non-compliance, enforce agreements, or build confidence that rules are being followed. As a result, some governance proposals may be much weaker in practice than they appear.

By contrast, if it were possible to infer what systems are doing from external signals, this could expand the set of viable approaches. Monitoring could become less dependent on internal access, and therefore more robust in settings where trust is limited.

More broadly, this reflects a general pattern in governance: *what can be reliably verified often shapes what can be effectively governed.*

The idea

The challenge, then, is to find a form of verification that does not depend on direct access to systems or model internals, and is still compatible with data security and real-world deployment constraints. One possible approach is to look for black-box signals generated by the hardware itself, which may be more resistant to tampering and can be observed without modifying the underlying computation. On this view, the question is whether large-scale training leaves a detectable footprint in GPU telemetry.

Modern GPUs expose a range of telemetry signals, such as power consumption, utilization, temperature, clock frequencies, and data transfer activity. These signals reflect the physical behavior of the device as it executes workloads. Because they arise from the hardware during execution, they may be harder to falsify than software logs or self-reports.

The core idea is that different types of workloads may produce systematically different patterns in these signals over time and this may give rise to a characteristic temporal structure in GPU telemetry.

If such signatures exist and are sufficiently distinct, it may be possible to infer, at least probabilistically, what kind of workload a GPU is running, without needing direct access to the system's internal state. This could provide a form of external, non-intrusive monitoring that complements existing governance approaches.

At the same time, this idea is uncertain and remains exploratory. It is not clear whether the differences between workloads are strong enough to be detected reliably in practice, especially in more complex or adversarial settings.

What telemetry might reveal

Different workloads do not just use different amounts of compute; they organize work differently, based on their architecture and data flow. As a result, even when overall utilization is similar, the temporal structure of execution may differ.

Training neural networks typically involves repeated cycles of forward passes, backpropagation, and optimization steps. While this structure is shared across many models, the way it is instantiated — including batching, data movement, and scheduling — can vary across workloads.

If these differences affect how computation is executed over time, they may be reflected in hardware-level signals. This would mean that telemetry could provide information not only about how much compute is used, but about how it is being used — which is precisely what verification requires.

Exploring the hardware signal patterns across workloads

Modern GPUs expose a continuous stream of low-level telemetry while they operate: power draw, utilization, temperature, clock behavior, and data transfer activity. These measurements are generated by the hardware itself and can be collected without accessing or modifying the model, the training code, or the data pipeline. Here, they were recorded using a C++ sampler that queries the GPU in real time, logging its physical state as different workloads execute.

If verification is to happen externally and without cooperation, these signals are one of the few sources of evidence available. They are not designed for interpretability, but they are an unavoidable byproduct of computation — a physical trace left on the device.

A natural starting point is GPU power consumption. Power reflects how much electrical energy the device is drawing and therefore how intensively it is being used. On its own, it does not reveal what computation is being performed. However, it may capture how that computation is organized over time.

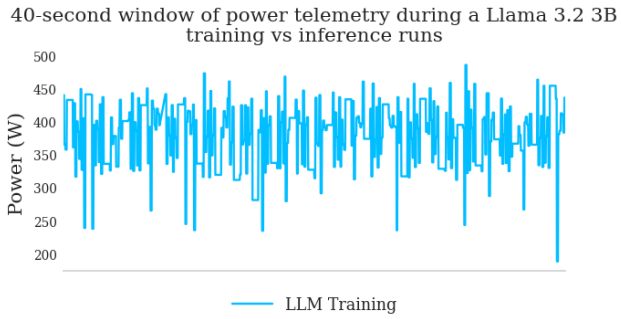

Figure 1: Power telemetry during LLM training. A 40-second window from a Llama 3.2 3B training run on an A100.

Figure 1 shows a 40-second window from an LLM training run. The signal remains at consistently high levels, with dense, continuous variation and no clear separation into idle periods.

For this pattern to be useful in a governance context, it must help distinguish training from other workloads that produce similar levels of activity. The same hardware and model may also be used for inference, where the model is executed rather than updated.

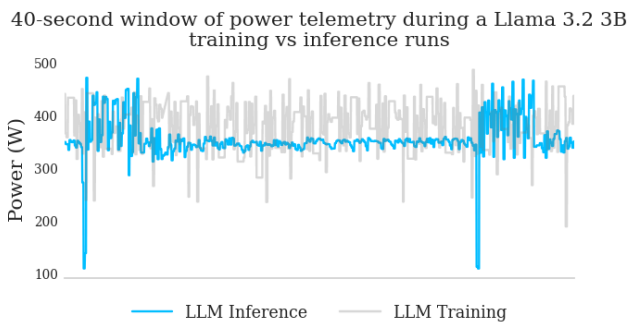

Figure 2: Power telemetry during LLM inference (blue) and LLM training (gray) runs on the same A100.

Figure 2 compares LLM training and inference. Both operate at high power levels, but differ in how that activity is distributed over time. Inference evolves more slowly, with broader transitions, while training exhibits tighter and more frequent variation.

This comparison addresses one important distinction between stages of use, but the broader challenge is not limited to training versus inference. It is whether telemetry can help separate LLMs from different kinds of workloads more generally, including those built on very different architectures.

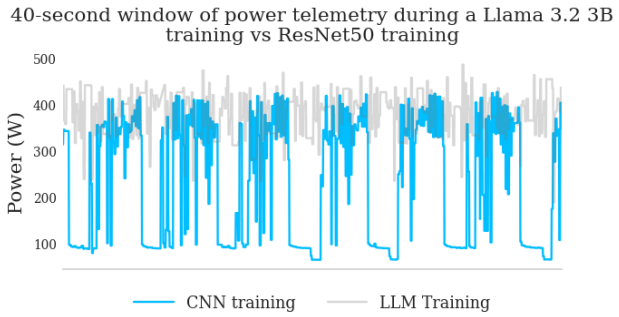

Figure 3: Power telemetry during CNN training. A 40-second window from a ResNet50 image-classification training run on the same A100.

Figure 3 extends the comparison to CNN training using ResNet50. While the overall power level remains high, the temporal structure changes, variation appears in more distinct and repeated cycles, suggesting a different organization of computation over time.

This indicates that workloads built on different architectures can produce visibly different patterns, even when they operate at similar levels of intensity. However, many modern systems are converging toward similar architectural foundations. Transformer-based models are now used across a wide range of domains, often relying on comparable computational primitives and data structures. This raises a more difficult question of whether telemetry can still distinguish workloads when their underlying structure is more similar.

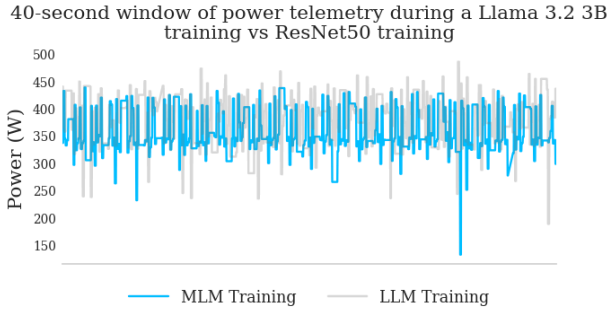

Figure 4: Power telemetry during MLM training. A 40-second window from a BERT-Large MLM training run on the same A100.

Figure 4 compares LLM training with BERT-Large Masked Language Modeling. Both rely on transformer architectures and operate over text, but they differ in their training objectives: while one predicts the next token in a sequence, the other reconstructs hidden tokens within the input.

At this level, the signals become harder to separate. Both traces occupy a similar high-power range and exhibit dense, continuous variation. The patterns begin to overlap, suggesting that while simple temporal signals can identify broad categories of compute, distinguishing between models within the same architectural family may not be straightforward.

To go further, the same workloads can then be viewed across multiple telemetry signals. While power draw is a high-fidelity proxy for intensity, it captures only one dimension of the GPU's state. The physical footprint of a workload is actually a coordinated event across several hardware layers: as the cores execute instructions (utilization), they draw energy (power), generate heat (temperature), and require a constant stream of information from the rest of the system (PCIe data transfer).

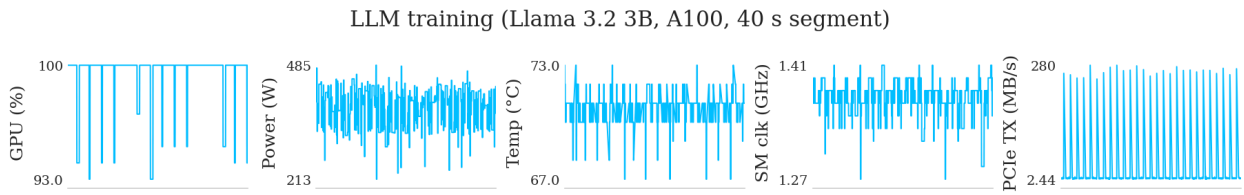

Figure 5. Telemetry during LLM training across multiple signals. A 40-second window showing GPU utilization, power, temperature, SM clock, and PCIe TX.

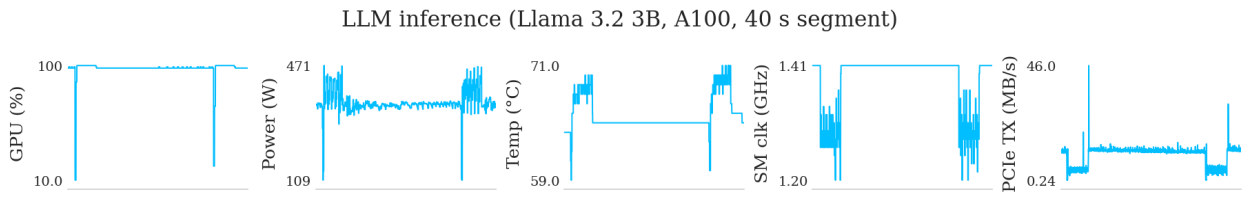

Figure 6. Telemetry during LLM inference across multiple signals. A 40-second window showing GPU utilization, power, temperature, SM clock, and PCIe TX.

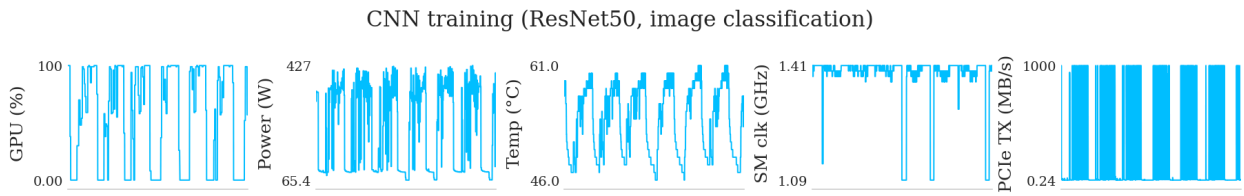

Figure 7. Telemetry during CNN training across multiple signals. A 40-second window showing GPU utilization, power, temperature, SM clock, and PCIe TX.

Figures 5, 6, and 7 show these signals for LLM training, LLM inference, and CNN training. When viewed together, they move in synchrony, reflecting the structure of each workload. Changes in power are accompanied by corresponding changes in utilization and data transfer, indicating that these patterns are system-level effects rather than isolated measurements.

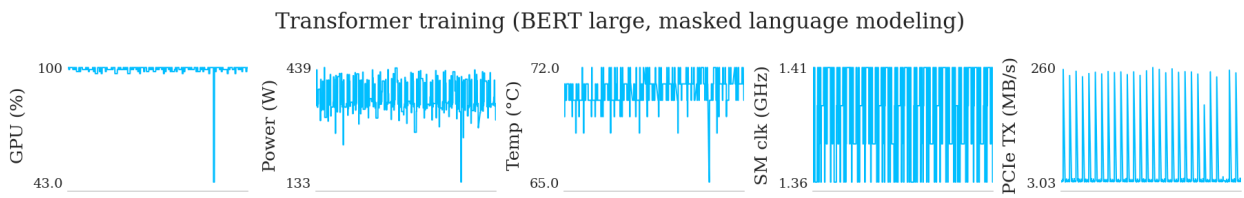

Figure 8. Telemetry during masked language modeling across multiple signals. A 40-second window showing GPU utilization, power, temperature, SM clock, and PCIe TX.

Figure 8 shows the same view for masked language modeling. While differences are subtle in power alone, they begin to emerge more clearly when multiple signals are considered together, suggesting that higher-dimensional telemetry may carry additional information about how computation is organized.

These observations remain qualitative. While the structure of different workloads is visible to the eye, this form of analysis is limited to visual inspection over short windows. To move beyond visual intuition, the next step is to characterize these patterns more precisely, identifying whether they can be measured and compared in a systematic way.

Characterizing signal structure in the frequency domain

Temporal traces provide an intuitive view of hardware activity, but they are often noisy and difficult to compare systematically. Small shifts in time, local fluctuations, or differences in observation windows can obscure whether a pattern is truly characteristic of a workload or just incidental.

To address this, the analysis can move to the frequency domain. Instead of asking how the signal evolves moment by moment, it becomes possible to ask how variation is distributed across timescales. Power spectral density provides a way to represent this, decomposing the signal into its underlying frequency components and revealing where most of the activity is concentrated.

This transformation makes signals more comparable. Rather than depending on the exact timing of events, it captures the structure of variation itself, allowing patterns to be summarized in a more stable and quantitative way.

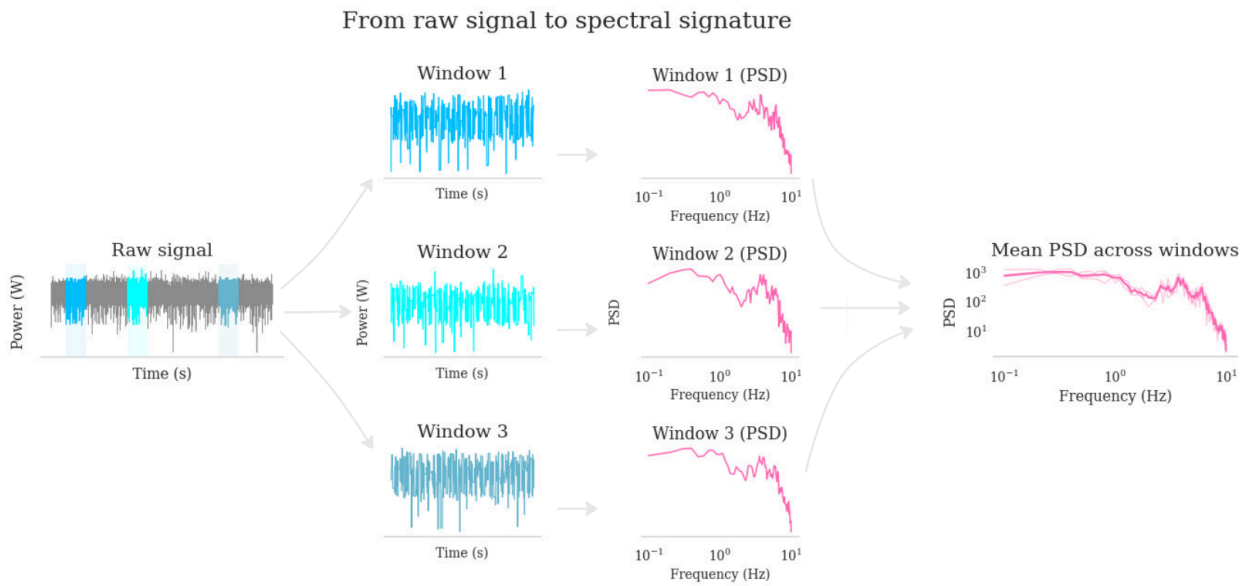

Figure 9. From raw signal to spectral signature. A telemetry trace is divided into windows; each window is converted into a power spectral density (PSD); the resulting spectra are then averaged.

Figure 9 shows how this representation is constructed. The original signal is divided into multiple windows, each window is transformed into a spectral representation, and the resulting spectra are averaged. The final result is a mean power spectral density that summarizes how signal variation is distributed across frequencies.

When this analysis is applied across workloads, some of the overlap seen in raw traces begins to reduce. Differences that are difficult to identify in the time domain become more apparent when viewed through their frequency composition.

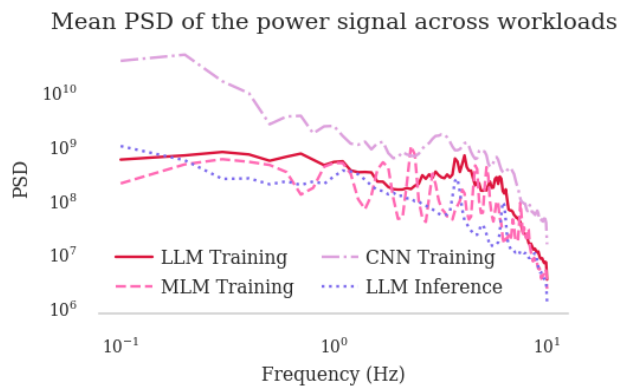

Figure 10. Mean power spectral density of the power signal across workloads.

Figure 10 shows the mean power spectral density (PSD) of the power signal across workloads. In this representation, the distinction between training and inference becomes clearer. LLM training and masked language modeling occupy similar ranges, but differ in how energy is distributed across frequencies. CNN training shifts toward lower-frequency components, suggesting a different temporal organization of computation.

These differences are not limited to power. Other telemetry signals provide additional structure, particularly in how computation interacts with data movement and system activity.

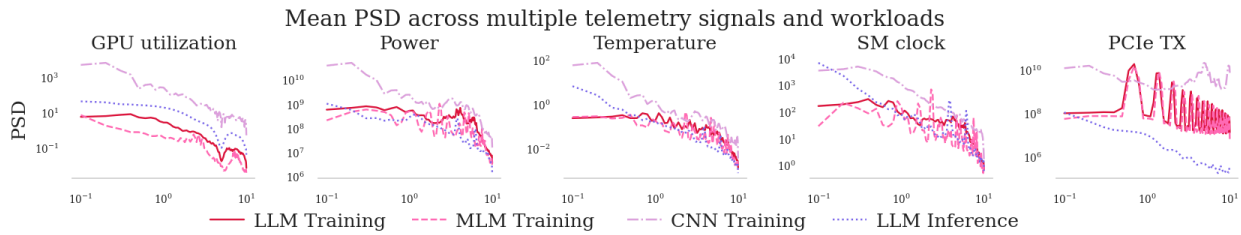

Figure 11. Mean PSD across multiple telemetry signals and workloads: GPU utilization, power, temperature, SM clock, and PCIe TX.

Figure 11 extends the analysis across multiple signals, including utilization, power, temperature, clock behavior, and PCIe transfer. In several cases, separation between workloads becomes more visible when these signals are considered together. Patterns that are subtle in one dimension can become clearer when viewed as part of a coordinated system response.

Taken together, these spectral representations suggest that the differences observed in the time domain are not only visual artifacts. They appear to reflect underlying differences in how workloads engage the device over time. By moving to the frequency domain, the analysis shifts from qualitative inspection toward a more systematic description of signal structure.

These results remain preliminary. Further work is needed to assess how robust these patterns are across different hardware, configurations, and workloads. Even so, they indicate that telemetry may carry information about how computation is being used, not just how much is being used. This points toward the possibility of using such signals as a complementary source of evidence for external monitoring in governance contexts.

What this does (and does not) show

The analysis above supports a limited but important conclusion: hardware telemetry appears to retain some information about how computation is organized over time.

This is more meaningful than it may initially seem. A natural skeptical view is that low-level GPU signals are too coarse or too noisy to reveal anything beyond whether a device is active. Once utilization is high, different workloads

might be expected to look indistinguishable from the outside. The examples above point in a different direction. Even under sustained load, signals do not collapse into a uniform pattern. Differences remain visible in how activity is distributed over time and how it interacts with data movement across the system.

At the same time, these results do not yet establish a reliable method for identifying workloads. They do not demonstrate robustness across different models, configurations, or hardware environments, nor do they address more complex or adversarial settings. This is not yet a classifier, a benchmark, or a deployable monitoring tool.

What they do suggest is narrower, but still useful. Hardware telemetry may contain more than a simple measure of intensity. It appears to preserve aspects of temporal organization that are tied to the underlying computation itself.

This matters because the governance problem is not only about measuring how much compute is used, but also about understanding how it is used. If low-level signals carry even partial information about workload type, they may provide a complementary source of evidence in settings where internal access is limited.

In that sense, the value of these results is not that they solve the verification problem, but that they make a more tractable version of it plausible. Instead of requiring precise identification, telemetry may contribute probabilistic signals, partial verification, or anomaly detection.

This is still far from a complete solution, but it suggests a different way of thinking about the problem. If computation leaves structured physical traces in the hardware that executes it, then external verification may not be out of reach.

Limitations

The analysis presented here is based on a narrow and highly controlled set of experiments. They are useful for illustrating what telemetry might reveal, but they leave many of the most important questions unresolved.

The scope of the experiments is limited. Each workload is represented by a single run, on a single hardware configuration, under a specific set of parameters. There is no variation in model size, dataset, sequence length, batch size, or implementation beyond what was required to produce a stable signal. As a result, it is not possible to assess how stable the observed patterns are under even modest changes to the workload. Signatures that appear distinctive in one configuration may shift, weaken, or disappear in another.

More fundamentally, the results do not disentangle workload effects from system effects. Telemetry reflects the behavior of the entire execution stack, not just the model. Input pipelines, data loading, preprocessing, caching, and scheduling can all influence the observed signals. This is especially visible in the CNN case, where data movement appears to play a large role, but it is not unique to it. Even in transformer workloads, choices such as dataset packing, tokenization strategy, and batching can materially change how the GPU is exercised. Without systematically varying these factors, it is difficult to isolate which aspects of the signal are intrinsic to the computation itself.

The analysis also focuses on relatively clean, steady-state segments of execution. Startup effects, checkpointing, logging, and other interruptions were intentionally excluded or minimized. This makes the patterns easier to observe, but it also removes sources of variability that would be present in real-world settings. In practice, telemetry traces may be noisier, less stable, and harder to segment cleanly.

The spectral analysis introduces its own constraints. Power spectral density provides a compact way to summarize how variation is distributed across time scales, but it necessarily discards temporal ordering and phase information. Two signals with different structures can produce similar spectral profiles, and small changes in preprocessing or windowing can affect the resulting curves. The analysis here uses simple averaging across windows, without exploring alternative representations or robustness to different parameter choices.

Adversarial considerations are not addressed. All workloads were run in a cooperative setting, with no attempt to conceal or mimic specific patterns. In a governance context, this assumption may not hold. If telemetry-based monitoring were to become relevant, actors could have incentives to modify workloads, inject noise, or reshape execution patterns to evade detection. It is unclear how resistant these signals would be to even simple forms of obfuscation.

There is also no notion of decision thresholds or error rates. The results show that workloads can appear different under certain views, but they do not quantify how reliably they can be distinguished.

Finally, the hardware context is limited. All experiments were conducted on a single class of GPU, under a specific software stack. Telemetry signals depend on hardware characteristics such as power management, thermal behavior,

and clock scaling policies. It is not clear how well these patterns would transfer across different GPU architectures, drivers, or deployment environments.

Taken together, these limitations mean that the results should be interpreted as exploratory. They show that telemetry can exhibit structured differences across workloads, but they do not establish that those differences are stable, generalizable, or robust enough for real-world use.

What they do provide is a *starting point*. They suggest which aspects of the signal may be informative, where distinctions begin to emerge, and which parts of the problem are likely to require deeper investigation.

Open questions

The results above point to a number of open questions, many of which are more important than the initial observations themselves.

A central concern is stability. Do the patterns observed here persist across variations within the same workload? Changes in batch size, sequence length, dataset, or optimization strategy can substantially alter how computation is scheduled on the GPU. It remains unclear whether the temporal signatures seen here reflect intrinsic properties of the workload, or whether they are tightly coupled to specific configurations. If the latter is true, telemetry may be more useful for recognizing known setups than for identifying broader classes of computation.

Closely related is the question of invariance across environments. Telemetry signals are mediated by GPU architecture, driver behavior, and system-level policies such as clock scaling and thermal management. It is not obvious that a signature observed on one GPU (e.g. an A100) would transfer cleanly to another (e.g. H100), or even across software versions. Limited portability could significantly constrain the use of telemetry in heterogeneous real-world settings.

Separability between similar workloads may be the most practically important challenge. Distinguishing between clearly different regimes—such as CNN training and LLM inference—is less relevant than distinguishing between workloads that are structurally similar. For example, can different forms of transformer training be reliably separated? Can smaller-scale training be distinguished from frontier-scale runs? Can inference be differentiated from fine-tuning or continued pretraining? While the results suggest that differences may exist, their strength and consistency remain uncertain.

Another unresolved issue is the role of system-level effects. Some of the clearest patterns appear to arise not only from computation itself, but from its interaction with data movement and input pipelines. This raises a deeper question: is telemetry capturing the intrinsic structure of the workload, or the structure of the system executing it? While this distinction may not matter for detection in practice, it has important implications for generalization and robustness.

Adversarial robustness remains largely unexplored. If telemetry-based monitoring were used in governance, actors could have incentives to alter execution patterns, inject noise, or otherwise reshape signals to resemble benign workloads. Some aspects of telemetry—particularly those tied to physical constraints like power and thermals—may be harder to manipulate precisely. Others, such as utilization or data transfer patterns, may be more flexible. Understanding which features are robust to intentional manipulation is likely to be critical.

There is also the question of how to combine signals effectively. Different telemetry channels appear to capture different aspects of execution, and it is unlikely that any single signal is sufficient on its own. Useful distinctions may instead emerge from combining multiple weak signals. This raises methodological questions around feature construction, representation learning, and dimensionality reduction, as well as practical questions about which signals are consistently available across environments.

Finally, there is a question of what level of evidence is sufficient for governance. Even if telemetry cannot provide perfect identification of workloads, it may still be valuable if it provides partial or probabilistic evidence. It could support anomaly detection, auditing, or confidence-building mechanisms in low-trust settings. This suggests that the relevant goal may not be exact classification, but the ability to detect patterns that are unlikely under benign or declared usage.

Taken together, these questions shift the focus of the problem. The key issue is not simply whether telemetry contains information, but whether that information can remain stable, transferable, and robust under the conditions that matter for real-world use.

Appendix: Data collection

- Telemetry was collected on a single NVIDIA A100-SXM4-40GB GPU.
- LLM training runs used Llama 3.2 3B, fine-tuned on a subset of OpenWebText in a single-device setup. Inference runs used the same model under high-throughput batched generation to maintain sustained GPU utilization.
- Additional workloads included ResNet50 training on the Imagenette dataset and masked language modeling using a transformer model trained on the same OpenWebText subset.
- Telemetry was recorded using a custom sampler built on NVIDIA's NVML interface, with a target sampling interval of 10 ms and an effective rate of approximately 30–40 Hz. The signals collected include GPU utilization, power consumption, temperature, SM and memory clock frequencies, and PCIe transmit throughput.
- Data was logged continuously during execution and segmented offline. Analysis focused on steady-state regions, excluding initialization phases and interruptions where possible.
- All experiments were conducted on a single GPU under controlled conditions, without checkpointing or concurrent workloads.
- Code for data collection and analysis is available at: https://github.com/AlmeidaAlin3/The-Verification-Problem-in-Compute-Governance .